IT112 Web System Development

Phase 5 – Displaying and Managing Data on the Frontend

Lab Report by TechNet

Group Members:

Mico Malana

Christian Sannie Silo

Jay Lloyd Chicogo

John Nelson Andes

Allen Benedict Asuncion

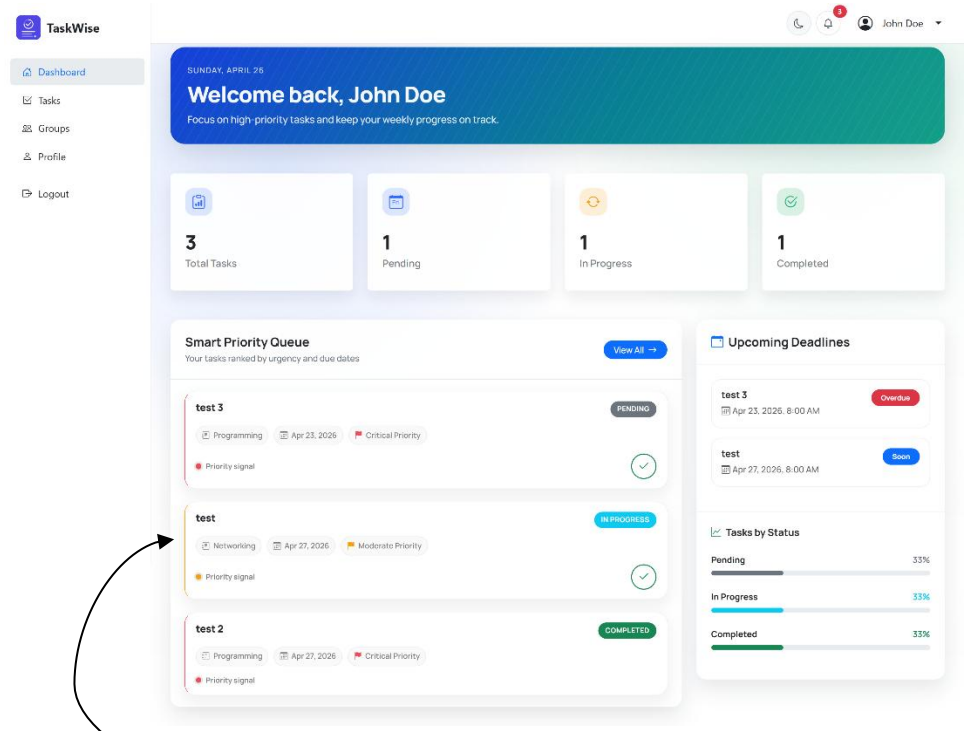
Instructor:

Guillermo V. Red, Jr., DIT

I. Display Pages

Below are screenshots of the display pages and sample API responses from Postman.

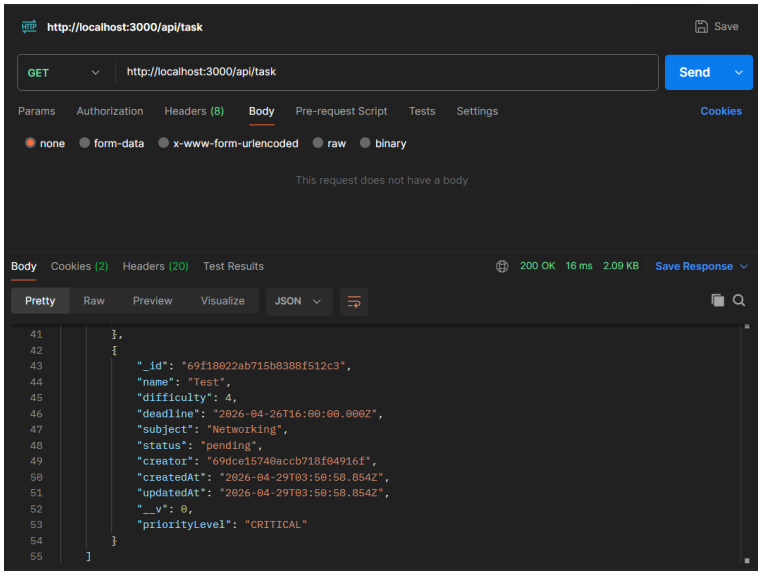
a. Dashboard Page



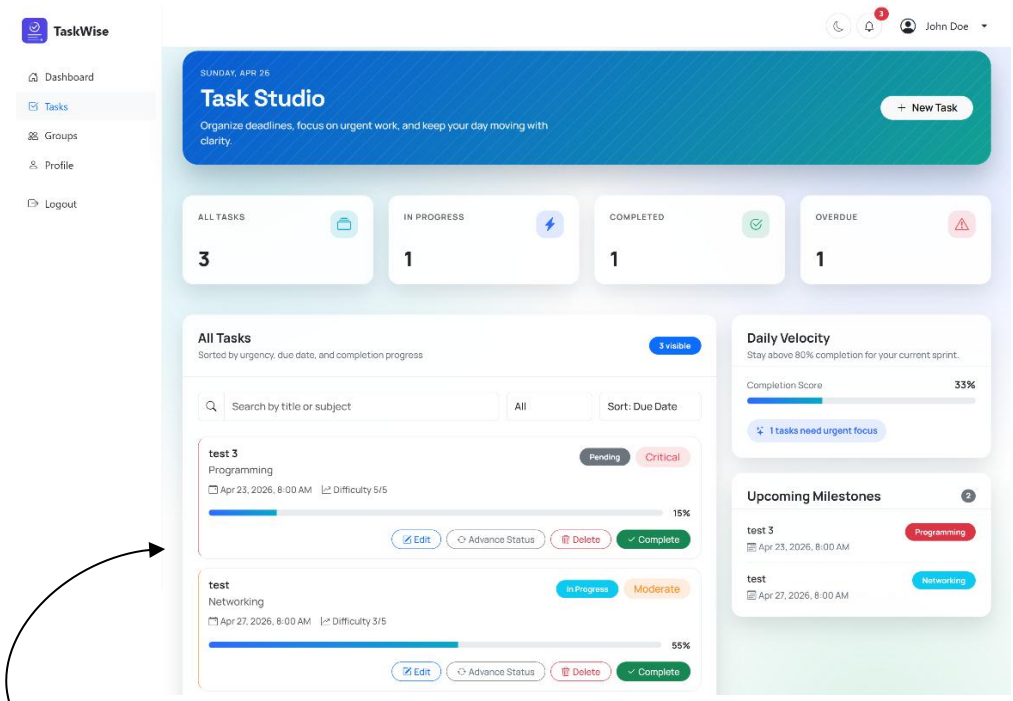
Here, the Smart Task Queue displays all tasks along with their corresponding priority levels, due dates, and progress status. The tasks are ordered based on their due dates and level of difficulty.

Sample API response using Postman:

GET api/task



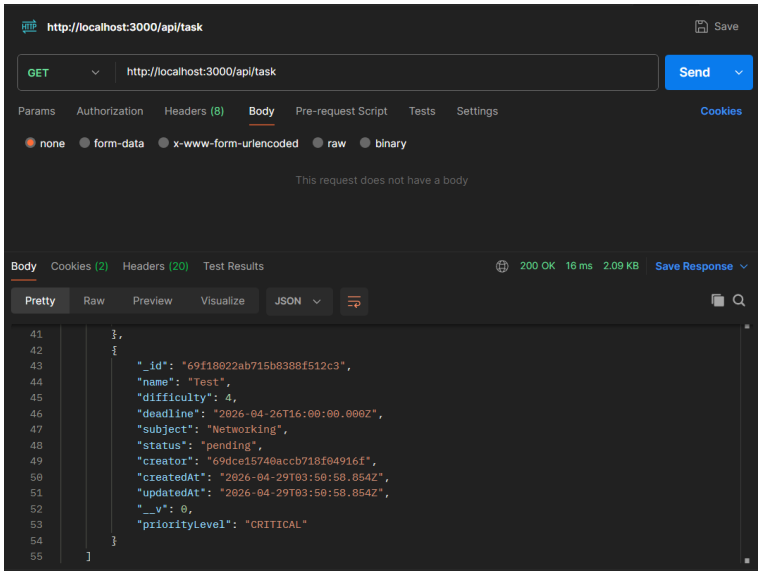
b. **Tasks Page**



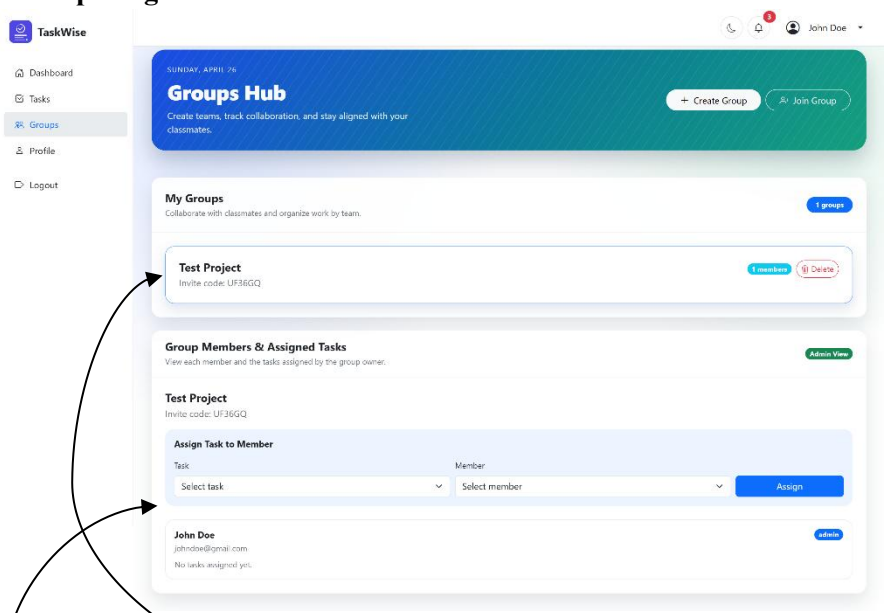
Here, the Tasks page lists all tasks along with their corresponding priority levels, due dates, and progress status.

Sample API response using Postman:

GET api/task



c. Groups Page

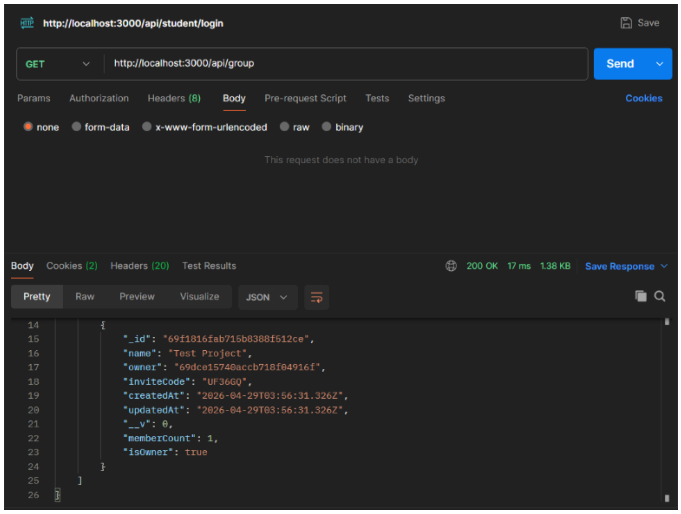


Here, the Group List displays each group along with its invite code and member count.

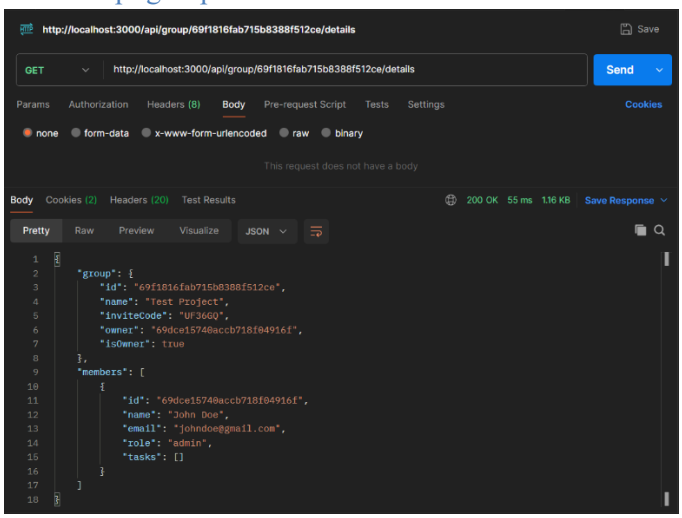
When a group is selected, it displays the members' names along with their assigned tasks.

Sample API response using POSTMAN:

GET api/group



GET api/group/:id/details



II. Annotated JavaScript code snippets for rendering

The following snippets show how data is fetched, processed, and rendered on the frontend, with annotations explaining each part.

a. Dashboard Rendering

We first destructure the data and functions returned by the useDashboardData hook, which handles fetching and managing all dashboard-related data:

```
const {
  student,
  taskItems,
  stats,
  deadlines,
  progress,
  loading,
  error,
  completeTask,
} = useDashboardData(handleUnauthorized);
```

Here, we render the stats data using map() function. Each item in stats array is passed to the StatCard component for display:

```
<Row className="g-3 g-xl-4 mb-4 mb-xl-5">
  {stats.map((stat : { id: string; icon: string;…) : Element => (
    <Col key={stat.id} xl={3} lg={6} md={6} sm={12}>
      <StatCard {...stat} />
    </Col>
  ))}
</Row>
```

Here, the StatCard component destructures the props passed to it in order to display the statistical data

```
import { Card } from "react-bootstrap";

const StatCard = ({ icon, accent, value, label } : { icon: any; accent: any; v…) : Element => {
  return (
    <Card className="stat-card border-0 h-100">
      <Card.Body className="p-4">
        <div className="d-flex align-items-center justify-content-between mb-4">
          <span className="stat-card-icon" style={{ color: accent }}>
            <i className={`bi bi-${icon}`}></i>
          </span>
        </div>

        <h3 className="stat-card-value mb-1">{value}</h3>
        <p className="stat-card-label mb-2">{label}</p>
      </Card.Body>
    </Card>
  );
};

export default StatCard;
```

b. Tasks Rendering

We first destructure the useTasksData hook, which provides the task data and functions needed for managing tasks in the application:

```
const {
  loading,
  error,
  stats,
  completion,
  upcomingMilestones,
  visibleTasks,
  query,
  setQuery,
  statusFilter,
  setStatusFilter,
  sortBy,
  setSortBy,
  completeTask,
  cycleTaskStatus,
  removeTask,
  addTask,
  editTask,
} = useTasksData(handleUnauthorized);
```

Then, we pass all the necessary data to the TaskList component, which is responsible for rendering the list of tasks and handling task-related actions:

```
<TaskList
  loading={loading}
  tasks={visibleTasks}
  query={query}
  onQueryChange={setQuery}
  statusFilter={statusFilter}
  onStatusFilterChange={setStatusFilter}
  sortBy={sortBy}
  onSortByChange={setSortBy}
  onEdit={handleOpenEdit}
  onComplete={completeTask}
  onCycleStatus={cycleTaskStatus}
  onDelete={removeTask}
/>
```

Here, inside the TaskList, we render each task dynamically using the map() function and pass its data to the TaskItem component. Each task also includes handlers for actions such as edit, complete, update status, and delete:

```
<div className="tasks-list-items">
  {tasks.map((task : any) : Element => (
    <TaskItem
      key={task.id}
      {...task}
      onEdit={() : any => onEdit(task)}
      onComplete={() : any => onComplete(task.id)}
      onCycleStatus={() : any => onCycleStatus(task.id, task.rawStatus)}
      onDelete={() : any => onDelete(task.id)}
    />
  ))}
</div>
```

c. Groups Rendering

We first destructure the data and functions returned by the useGroupsData hook, which handles fetching, managing, and updating all group-related data:

```
const {
  groups,
  loading,
  error,
  reload,
  groupDetails,
  detailsLoading,
  detailsError,
  assignableTasks,
  addGroup,
  joinExistingGroup,
  deleteGroupById,
  leaveSelectedGroup,
  removeMemberFromGroup,
  assignTask,
} = useGroupsData(handleUnauthorized, selectedGroupId);
```

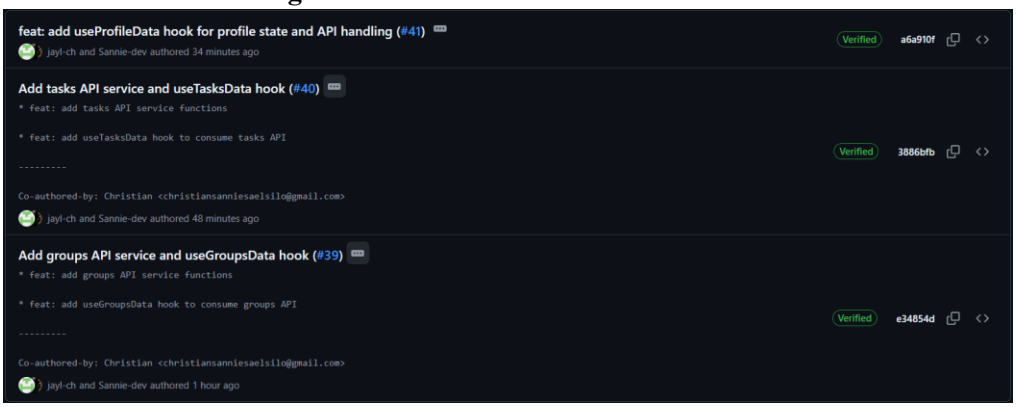
We then pass the destructured data to the GroupList component, which receives the group data and functions for managing groups. It handles displaying all groups, s electing a group, reloading data, and performing actions such as deleting or leaving a group:

```
<GroupList
  groups={groups}
  loading={loading}
  error={error}
  onReload={reload}
  onDelete={(groupId : any) : Promise<void> => {     • This may be converted to an async function.
    const selectedGroup = groups.find((group : { id: any; name: any; invit…) : boolean => group.id === groupId);

    if (selectedGroup?.isOwner) {
      return handleDeleteGroup(groupId);
    }

    return leaveSelectedGroup(groupId)
      .then(() : void => {
        if (groupId === selectedGroupId) {
          navigate(to: "/groups");
        }
      })
      .catch((err : any) : void => {
        setActionError(err?.response?.data?.message || "Failed to leave group.");
      });
  }}
  selectedGroupId={selectedGroupId}
  onSelect={(groupId : any) : void | Promise<void> => navigate(to: `/groups/${groupId}`)}
/>
```

Here, we show how each group member is rendered dynamically using the map() function. It also displays the tasks assigned to each member, or shows a fallback message if no tasks are available:

```
{member.tasks?.length ? (
  <div className="d-grid gap-2 mt-2">
    {member.tasks.map((task : any) : Element => (
      <div key={task.assignmentId} className="member-task-item rounded-3 p-2">
        <div className="fw-semibold small mb-1">{task.taskName}</div>
        <div className="d-flex gap-2 flex-wrap">
          <Badge bg="light" text="dark" pill>
            {task.subject}
          </Badge>
          <Badge
            bg={
              task.status === "completed"
                ? "success"
                : task.status === "in-progress"
                  ? "info"
                  : "secondary"
            }
            pill
          >
            {task.status}
          </Badge>
        </div>
      </div>
    ))}
  </div>
) : (
  <p className="small text-secondary mb-0">No tasks assigned yet.</p>
)}
```

III. GitHub Commits Logs



IV. Member Reflections

- **Mico Malana (Project Manager, Documentation Officer)**
  As the Project Manager my job was to assign the responsibilities and monitor the progress of each of my group members given task. Also, to make sure that their tasks progress align with our project. As the Documentation Officer my job is to document each progress, take screenshots and to explain their functionality and create a PDF file containing all of the week 14 progress.

- **Christian Silo (Frontend Developer)**
  I worked as a Frontend Developer for Team Innovators. My job was to handle data rendering and build the edit and delete features for our application. I had to make sure the edit and delete features for our application were working properly. Our team used Messenger GC to talk to each other all the time. We did this to solve problems with how things worked and to fix problems that were hard to find like the ones we had with the delete feature for our application.

- **Jay Lloyd Chicogo (Github Manager, Backend Developer)**
  I was assigned to help connect the frontend to the backend so the data is displayed correctly on the UI. I also made sure the repository stays organized by requiring pull requests before merging to the main branch.

- **John Nelson Andes (Database Manager)**
  As a Database Manager, I was responsible for managing the database collections and ensuring that the data is correctly rendered on the frontend. I made sure that the stored data is consistent and properly connected to the system.

- **Allen Benedict Asuncion (Tester & Debugger)**
  As the Tester, I was assigned to check the interaction of the frontend to see if it responds correctly to user actions. I tested if all interactive components are working as expected and properly rendered for the user. I also verified that user inputs and actions produce the correct results in the system.